

Portfolio Optimization using Stochastic Portfolio Theory

Contents

1	Markowitz Portfolio Optimization	4
1.1	Portfolio Weights	4
1.2	Expected Return Vector	5
1.3	Covariance Matrix	5
1.4	Portfolio Variance	5
1.5	Sharpe Ratio Objective	6
1.6	Optimization Procedure	6
1.7	Rolling Window Framework	7
1.8	Mathematical Formulation	8
1.9	Advantages of Rolling Markowitz	8
1.9.1	Adaptation to Market Regimes	8
1.9.2	Dynamic Covariance Estimation	9
1.9.3	Improved Risk Management	9
1.9.4	Reduced Model Staleness	9
1.10	Key Difference Between Static and Rolling Markowitz	9
2	Machine Learning Based Portfolio Optimization	10
2.1	Feature Engineering	10
2.1.1	Momentum Features	10
2.1.2	Volatility Feature	10
2.1.3	Moving Average Ratio Features	11
2.1.4	Relative Strength Index (RSI)	11
2.1.5	Mean Reversion Feature	11
2.2	Feature Tensor Construction	11
2.3	Cross-Sectional Normalization	12
2.4	Neural Network Architecture	12
2.5	Portfolio Weight Generation	13
2.6	Portfolio Return	13
2.7	Sharpe Ratio Objective Function	13
2.7.1	Portfolio Return Computation	14
2.7.2	Expected Portfolio Return	14
2.7.3	Portfolio Volatility	15
2.7.4	Sharpe Ratio Term	15
2.7.5	L2 Regularization Term	15

2.7.6	Maximum Drawdown Penalty	15
2.8	Batch Size and Full Batch Training	16
2.9	Optimization Procedure	17
2.10	Monthly Rebalancing	18
2.11	Position Constraints	18
2.12	Real World Portfolio Controls	18
2.12.1	Volatility Targeting	18
2.12.2	Drawdown Exposure Reduction	18
2.13	Key Difference from Markowitz Optimization	19
3	Stochastic Portfolio Theory Based Portfolio Optimization	19
3.1	Market Weight Construction	19
3.2	Diversity Weighted Portfolio	20
3.3	Interpretation of Parameter p	20
3.3.1	Case: $p = 1$	20
3.3.2	Case: $0 < p < 1$	21
3.4	Portfolio Weight Generation Procedure	21
3.5	Portfolio Returns	21
3.6	Market Portfolio Returns	22
3.7	Cumulative Wealth	22
3.8	Relative Wealth	22
3.9	Performance Metrics	23
3.9.1	Annualized Arithmetic Return	23
3.9.2	Annualized Volatility	23
3.9.3	Sharpe Ratio	24
3.9.4	Geometric Return	24
4	Parametric Extension of Stochastic Portfolio Theory	24
4.1	Learnable Diversity Parameter	25
4.2	Relative Return Objective	25
4.3	Relative Sharpe Ratio Optimization	25
4.4	Gradient-Based Learning of p	26
4.5	Why This Improves Over Classical SPT	27
4.6	Key Difference from Previous SPT Model	27
5	Functional Stochastic Portfolio Theory (Functional SPT)	27
5.1	Motivation and Key Improvement over Previous Models	27
5.2	Core Idea of Functional SPT	28
5.3	Feature Construction	28
5.3.1	Risk-Adjusted Momentum	29
5.4	Cross-Sectional Ranking	29
5.5	GARCH Volatility Estimation	29
5.6	Recursive Volatility Update	30
5.7	Neural Network Portfolio Generator	30
5.8	Residual Connection	30

5.9	Softplus Activation	31
5.10	Final Portfolio Weights	31
5.11	GARCH Risk Adjustment	31
5.12	Objective Function	31
5.13	Relative Log-Wealth Objective	32
5.14	Sharpe Ratio Component	33
5.15	Downside Risk Penalty	33
5.16	Concentration Penalty	34
5.17	Drawdown Penalty	34
5.18	Smoothness Regularization	34
5.19	Gradient-Based Optimization	35
5.20	Gradient Clipping	35
5.21	Learning Rate Scheduling	35
5.22	Early Stopping	35
5.23	Monthly Rebalancing	36
5.24	Key Improvements over Previous SPT Models	36
5.25	Main Conceptual Difference	36
6	Transaction Cost Adjustment	36
6.1	Portfolio Turnover	37
6.2	Transaction Cost Model	37
6.3	Net Portfolio Return	37
6.4	Rebalancing Frequency	38
6.5	Impact on Portfolio Performance	38

1 Markowitz Portfolio Optimization

The objective of Markowitz Portfolio Optimization is to determine the optimal portfolio weights that maximize the Sharpe Ratio while considering both expected return and portfolio risk.

The optimization problem is defined as:

$$\max_w \frac{w^T \mu}{\sqrt{w^T \Sigma w}}$$

subject to:

$$\sum_{i=1}^N w_i = 1$$

and

$$0 \leq w_i \leq 0.4$$

where:

- w = portfolio weight vector
- μ = expected return vector
- Σ = covariance matrix

1.1 Portfolio Weights

The portfolio weights are represented as:

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}$$

where w_i denotes the fraction of total capital allocated to stock i .

The constraint:

$$\sum_{i=1}^N w_i = 1$$

ensures that all available capital is fully invested.

The bound constraint:

$$0 \leq w_i \leq 0.4$$

prevents excessive concentration in a single stock.

1.2 Expected Return Vector

The expected return vector is:

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_N \end{bmatrix}$$

where μ_i represents the estimated expected return of stock i .

In this project, expected returns are estimated using Exponentially Weighted Moving Average (EMA), which assigns greater importance to recent observations.

The portfolio expected return is computed as:

$$\mu_p = w^T \mu$$

or equivalently:

$$\mu_p = \sum_{i=1}^N w_i \mu_i$$

1.3 Covariance Matrix

The covariance matrix captures both individual asset volatility and relationships between assets.

$$\Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \cdots & \sigma_{1N} \\ \sigma_{21} & \sigma_{22} & \cdots & \sigma_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{N1} & \sigma_{N2} & \cdots & \sigma_{NN} \end{bmatrix}$$

where:

- σ_{ii} = variance of asset i
- σ_{ij} = covariance between assets i and j

The covariance matrix is critical because it enables the optimizer to measure diversification benefits and joint portfolio risk.

1.4 Portfolio Variance

Portfolio variance is computed as:

$$\sigma_p^2 = w^T \Sigma w$$

Expanded form:

$$\sigma_p^2 = \sum_{i=1}^N \sum_{j=1}^N w_i w_j \sigma_{ij}$$

This formulation captures:

- individual asset risk
- correlation between assets
- diversification effects

Portfolio volatility is:

$$\sigma_p = \sqrt{w^T \Sigma w}$$

1.5 Sharpe Ratio Objective

The Sharpe Ratio measures return per unit of risk.

$$S(w) = \frac{w^T \mu}{\sqrt{w^T \Sigma w}}$$

The optimizer attempts to:

- maximize expected return
- minimize portfolio volatility

simultaneously.

1.6 Optimization Procedure

The optimization algorithm iteratively adjusts portfolio weights in directions that improve the Sharpe Ratio while satisfying portfolio constraints.

The optimization process proceeds as follows:

1. Estimate expected returns μ
2. Estimate covariance matrix Σ
3. Initialize portfolio weights
4. Compute portfolio return and volatility
5. Evaluate Sharpe Ratio
6. Compute the direction that improves the objective function
7. Update portfolio weights

8. Repeat until convergence

The optimizer searches for the weight vector that produces the best trade-off between return and risk.

Conceptually, the weight update process can be represented as:

$$w^{(k+1)} = w^{(k)} + \eta \nabla S(w)$$

where:

- $w^{(k)}$ = current portfolio weights
- η = step size or learning rate
- $\nabla S(w)$ = gradient direction of the Sharpe Ratio

The gradient indicates how the portfolio weights should change in order to improve the Sharpe Ratio.

If increasing the weight of a stock improves the objective function, the optimizer increases its allocation. Conversely, if a stock contributes excessive risk relative to return, its allocation is reduced.

The optimization continues iteratively until the Sharpe Ratio can no longer be improved significantly while satisfying all portfolio constraints.

Although the optimization library internally uses convex optimization solvers rather than standard neural-network gradient descent, the iterative process conceptually behaves as repeated weight updates in directions that improve the objective function.

The covariance matrix plays a central role because it allows the optimizer to reduce total portfolio risk by allocating capital across less correlated assets.

1.7 Rolling Window Framework

In Rolling Markowitz optimization, the model repeatedly retrains using a moving historical window.

At each rebalance date:

1. Select recent historical data window
2. Estimate expected returns
3. Estimate covariance matrix
4. Solve optimization problem
5. Obtain optimal portfolio weights
6. Apply weights to next holding period
7. Move window forward

8. Repeat process

If the rolling window length is 252 trading days:

$$\text{Training Window} = 252$$

then the optimizer only uses the most recent one year of market data to estimate parameters.

1.8 Mathematical Formulation

At rebalance time t :

μ_t = Expected Returns estimated using recent window

Σ_t = Covariance Matrix estimated using recent window

The optimization problem becomes:

$$\max_{w_t} \frac{w_t^T \mu_t}{\sqrt{w_t^T \Sigma_t w_t}}$$

subject to:

$$\sum_i w_{t,i} = 1$$

and

$$0 \leq w_{t,i} \leq 0.4$$

The optimal weights are recalculated at every rebalance period.

1.9 Advantages of Rolling Markowitz

Rolling optimization provides several important advantages over static optimization.

1.9.1 Adaptation to Market Regimes

Market conditions continuously evolve:

- bull markets
- bear markets
- volatility spikes
- sector rotations

Rolling optimization allows portfolio weights to adapt dynamically to recent market behavior.

1.9.2 Dynamic Covariance Estimation

Correlations between assets are not constant over time.

Rolling covariance estimation enables the optimizer to capture:

- changing relationships between assets
- increasing systemic risk
- sector concentration risk

more effectively than static optimization.

1.9.3 Improved Risk Management

Because portfolio weights are updated periodically, the portfolio can:

- reduce exposure to deteriorating assets
- increase allocation to improving assets
- respond to changing volatility regimes

This improves risk management compared to fixed-weight portfolios.

1.9.4 Reduced Model Staleness

Static optimization assumes historical relationships remain stable indefinitely.

This assumption is unrealistic in financial markets.

Rolling optimization reduces parameter staleness by continuously updating:

- expected returns
- covariance structure
- optimal allocations

using recent data.

1.10 Key Difference Between Static and Rolling Markowitz

Feature	Static Markowitz	Rolling Markowitz
Weights	Fixed	Dynamic
Covariance Matrix	Estimated Once	Updated Repeatedly
Expected Returns	Fixed Estimate	Continuously Updated
Market Adaptation	Low	High
Regime Sensitivity	Weak	Strong

Rolling Markowitz therefore provides a more realistic framework for practical portfolio management in dynamic financial markets.

2 Machine Learning Based Portfolio Optimization

Unlike classical Markowitz optimization, which directly solves a mathematical optimization problem using expected returns and covariance matrices, the machine learning based approach learns portfolio allocation patterns directly from historical market features.

The model attempts to learn a mapping:

$$X \rightarrow w$$

where:

- X = feature matrix
- w = portfolio weights

The objective is to learn portfolio weights that maximize the out-of-sample Sharpe Ratio while controlling drawdowns and model complexity.

2.1 Feature Engineering

Several market features are constructed from historical prices to capture momentum, volatility, trend-following, and mean-reversion behavior.

2.1.1 Momentum Features

Momentum measures recent price strength.

$$\text{Momentum}_5 = \frac{P_t}{P_{t-5}} - 1$$

$$\text{Momentum}_{20} = \frac{P_t}{P_{t-20}} - 1$$

$$\text{Momentum}_{60} = \frac{P_t}{P_{t-60}} - 1$$

where:

- P_t = current price

These features capture short-term, medium-term, and long-term trends.

2.1.2 Volatility Feature

Rolling volatility is estimated as:

$$\sigma_{20} = \text{StdDev}(r_t)$$

using a 20-day rolling window.

This measures recent market uncertainty and risk.

2.1.3 Moving Average Ratio Features

Trend-following signals are constructed using moving average ratios.

$$MA_{20} = \frac{P_t}{SMA_{20}}$$

$$MA_{50} = \frac{P_t}{SMA_{50}}$$

where:

- SMA_{20} = 20-day moving average
- SMA_{50} = 50-day moving average

Values above 1 indicate bullish trend behavior.

2.1.4 Relative Strength Index (RSI)

RSI measures momentum strength and overbought/oversold conditions.

$$RSI = 1 - \frac{1}{1 + RS}$$

where:

$$RS = \frac{\text{Average Gain}}{\text{Average Loss}}$$

2.1.5 Mean Reversion Feature

A contrarian signal is constructed as:

$$\text{Rev}_5 = - \left(\frac{P_t}{P_{t-5}} - 1 \right)$$

This feature attempts to capture short-term reversal behavior.

2.2 Feature Tensor Construction

For each trading day:

$$X_t \in \mathbb{R}^{N \times F}$$

where:

- N = number of stocks
- F = number of features

In this project:

$$F = 8$$

The complete dataset becomes:

$$X \in \mathbb{R}^{T \times N \times F}$$

where:

- T = number of trading days

2.3 Cross-Sectional Normalization

For each day, features are normalized across stocks:

$$X_{norm} = \frac{X - \mu_X}{\sigma_X}$$

This removes scale differences and allows the model to focus on relative cross-sectional information.

2.4 Neural Network Architecture

The portfolio model is implemented using a feedforward neural network.

The network learns a nonlinear scoring function:

$$f_{\theta}(X_i) \rightarrow s_i$$

where:

- X_i = feature vector of stock i
- s_i = predicted score
- θ = neural network parameters

The architecture contains:

- fully connected layers
- ReLU activation
- Batch Normalization
- Dropout regularization

The network structure is:

$$8 \rightarrow 64 \rightarrow 32 \rightarrow 1$$

2.5 Portfolio Weight Generation

The neural network outputs raw scores:

$$s_i$$

for each stock.

These scores are transformed into portfolio weights using the softmax function:

$$w_i = \frac{e^{s_i}}{\sum_{j=1}^N e^{s_j}}$$

Properties of softmax:

- all weights are positive
- weights sum to 1
- higher scores receive larger allocations

The final portfolio weight vector becomes:

$$w = [w_1, w_2, \dots, w_N]$$

2.6 Portfolio Return

Portfolio return at time t is computed as:

$$R_{p,t} = \sum_{i=1}^N w_{i,t} r_{i,t}$$

where:

- $r_{i,t}$ = return of stock i

2.7 Sharpe Ratio Objective Function

The model is trained to maximize the portfolio Sharpe Ratio.

The Sharpe Ratio is defined as:

$$S = \frac{\mathbb{E}[R_p]}{\sigma(R_p)}$$

where:

- $\mathbb{E}[R_p]$ = expected portfolio return
- $\sigma(R_p)$ = portfolio return volatility

Since deep learning frameworks minimize loss functions, the model minimizes the negative Sharpe Ratio.

The complete loss function is:

$$\mathcal{L} = -\frac{\mathbb{E}[R_p]}{\sigma(R_p)} + \lambda_{L2}||\theta||^2 + \lambda_{DD} \cdot DD_{max}$$

where:

- first term maximizes Sharpe Ratio
- second term applies L2 regularization
- third term penalizes maximum drawdown

2.7.1 Portfolio Return Computation

For each trading day t , the portfolio return is computed using predicted portfolio weights and realized stock returns:

$$R_{p,t} = \sum_{i=1}^N w_{i,t} r_{i,t}$$

where:

- $w_{i,t}$ = portfolio weight of stock i
- $r_{i,t}$ = realized return of stock i
- N = number of stocks

The full portfolio return vector becomes:

$$R_p = [R_{p,1}, R_{p,2}, \dots, R_{p,T}]$$

where:

- T = total number of training days

2.7.2 Expected Portfolio Return

The expected portfolio return is estimated using the sample mean across the entire training dataset:

$$\mathbb{E}[R_p] = \frac{1}{T} \sum_{t=1}^T R_{p,t}$$

This corresponds to:

$$\text{mean} = \text{torch.mean}(R_p)$$

in the implementation.

The expected return measures the average profitability of the strategy.

2.7.3 Portfolio Volatility

Portfolio volatility is computed using the standard deviation of portfolio returns:

$$\sigma(R_p) = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - \mathbb{E}[R_p])^2}$$

Higher volatility implies larger fluctuations in portfolio performance.

2.7.4 Sharpe Ratio Term

Using expected return and volatility:

$$S = \frac{\mathbb{E}[R_p]}{\sigma(R_p)}$$

The model minimizes:

$$-S$$

which is equivalent to maximizing the Sharpe Ratio.

2.7.5 L2 Regularization Term

The second term penalizes excessively large neural-network parameters.

$$\|\theta\|^2 = \sum_k \theta_k^2$$

where:

- θ_k = neural-network parameter

The regularization term becomes:

$$\lambda_{L2} \sum_k \theta_k^2$$

This reduces overfitting and improves model generalization.

2.7.6 Maximum Drawdown Penalty

Portfolio cumulative wealth is computed as:

$$W_t = \prod_{i=1}^t (1 + R_{p,i})$$

Rolling maximum wealth:

$$M_t = \max(W_1, \dots, W_t)$$

Drawdown:

$$DD_t = \frac{M_t - W_t}{M_t}$$

Maximum drawdown:

$$DD_{max} = \max_t DD_t$$

The loss penalizes large drawdowns using:

$$\lambda_{DD} \cdot DD_{max}$$

This encourages more stable portfolio behavior during market stress.

2.8 Batch Size and Full Batch Training

The model uses full-batch optimization during training.

This means all training observations are used simultaneously when computing the loss function and gradients.

If:

$$X \in \mathbb{R}^{T \times N \times F}$$

then:

- T = number of trading days
- N = number of stocks
- F = number of features

The neural network processes all T trading days together during each optimization step. Therefore:

$$\text{Batch Size} = T$$

The Sharpe Ratio is computed using the complete portfolio return vector:

$$R_p = [R_{p,1}, R_{p,2}, \dots, R_{p,T}]$$

rather than using smaller mini-batches.

This is important because the Sharpe Ratio depends on global portfolio statistics:

- average return
- variance
- drawdown

Mini-batch training could produce unstable estimates of these quantities because each mini-batch would only contain partial portfolio history.

Full-batch optimization provides:

- stable Sharpe estimation
- stable variance estimation
- stable drawdown estimation
- smoother gradient updates

although it requires higher computational cost.

2.9 Optimization Procedure

The neural network parameters are optimized using the Adam optimizer.

The optimization process proceeds iteratively:

1. Input feature tensor into neural network
2. Generate stock scores
3. Convert scores into portfolio weights
4. Compute portfolio returns
5. Compute Sharpe-based loss function
6. Compute gradients using backpropagation
7. Update neural-network parameters
8. Repeat until convergence

The parameter update process conceptually follows gradient descent:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_{\theta} \mathcal{L}$$

where:

- θ = neural network parameters
- η = learning rate
- $\nabla_{\theta} \mathcal{L}$ = gradient of loss function

The gradients indicate how model parameters should change to improve portfolio performance.

Backpropagation computes these gradients automatically using the chain rule.

The Adam optimizer then updates the parameters using adaptive learning rates and momentum estimates.

2.10 Monthly Rebalancing

Although weights are predicted daily, the strategy applies monthly rebalancing.

The portfolio weights from the first trading day of each month are held constant for the next 21 trading days.

This reduces:

- excessive turnover
- transaction costs
- unstable daily trading

2.11 Position Constraints

Portfolio weights are constrained as:

$$0.02 \leq w_i \leq 0.20$$

This prevents:

- extremely small allocations
- excessive concentration

Weights are then renormalized to ensure:

$$\sum_i w_i = 1$$

2.12 Real World Portfolio Controls

Several practical controls are applied to improve robustness.

2.12.1 Volatility Targeting

Portfolio exposure is dynamically scaled to target annualized volatility:

$$\text{Scaling Factor} = \frac{\sigma_{target}}{\sigma_{rolling}}$$

This stabilizes portfolio risk across different volatility regimes.

2.12.2 Drawdown Exposure Reduction

If portfolio drawdown exceeds a threshold:

$$DD < -20\%$$

portfolio exposure is reduced.

This acts as a defensive risk-management mechanism during severe market stress.

2.13 Key Difference from Markowitz Optimization

The fundamental difference between Markowitz optimization and machine learning based optimization is the way portfolio weights are generated.

Feature	Markowitz	ML Based Model
Weight Generation	Closed-form optimization	Neural-network prediction
Inputs	Returns and covariance	Multi-factor features
Covariance Usage	Explicit	Indirect through training
Optimization Variable	Portfolio weights	Neural-network parameters
Learning Method	Convex optimization	Gradient-based learning
Nonlinearity	Limited	Strong nonlinear modeling

Unlike Markowitz optimization, the machine learning model does not explicitly solve a covariance optimization problem.

Instead, it learns allocation behavior directly from historical patterns using gradient-based optimization and nonlinear feature interactions.

3 Stochastic Portfolio Theory Based Portfolio Optimization

This portfolio construction approach is based on principles from Stochastic Portfolio Theory (SPT).

Unlike Markowitz optimization, which relies on expected returns and covariance matrices, SPT constructs portfolios directly using market weights.

The objective is to systematically exploit structural market properties such as:

- market diversification
- excess growth rate
- capital redistribution
- volatility pumping

without explicitly forecasting future returns.

3.1 Market Weight Construction

Since historical market capitalization data is unavailable, market weights are approximated using normalized stock prices.

For each trading day t :

$$\mu_{i,t} = \frac{P_{i,t}}{\sum_{j=1}^N P_{j,t}}$$

where:

- $P_{i,t}$ = price of stock i
- $\mu_{i,t}$ = approximate market weight
- N = total number of stocks

The market weight vector becomes:

$$\mu_t = [\mu_{1,t}, \mu_{2,t}, \dots, \mu_{N,t}]$$

The normalization ensures:

$$\sum_{i=1}^N \mu_{i,t} = 1$$

3.2 Diversity Weighted Portfolio

The portfolio is constructed using the Diversity Weighted Portfolio (DWP) formulation. Portfolio weights are defined as:

$$\pi_{i,t} = \frac{\mu_{i,t}^p}{\sum_{j=1}^N \mu_{j,t}^p}$$

where:

- $\pi_{i,t}$ = portfolio weight of stock i
- $\mu_{i,t}$ = market weight
- p = diversity parameter

The parameter p controls the portfolio concentration structure.

3.3 Interpretation of Parameter p

The diversity parameter determines how strongly the portfolio deviates from the market portfolio.

3.3.1 Case: $p = 1$

If:

$$p = 1$$

then:

$$\pi_{i,t} = \mu_{i,t}$$

and the portfolio becomes identical to the market portfolio.

3.3.2 Case: $0 < p < 1$

If:

$$0 < p < 1$$

then smaller-cap stocks receive relatively larger allocations compared to the market portfolio.

This creates a systematic diversification effect.

In this project:

$$p = 0.5$$

and

$$p = 0.8$$

are tested.

3.4 Portfolio Weight Generation Procedure

The portfolio weight generation process proceeds as follows:

1. Compute normalized market weights
2. Raise each market weight to power p
3. Sum all transformed weights
4. Normalize transformed weights
5. Obtain final portfolio weights

Mathematically:

$$\mu_{i,t} \rightarrow \mu_{i,t}^p \rightarrow \pi_{i,t}$$

The normalization step ensures:

$$\sum_{i=1}^N \pi_{i,t} = 1$$

3.5 Portfolio Returns

Portfolio return at time t is computed as:

$$R_{p,t} = \sum_{i=1}^N \pi_{i,t-1} r_{i,t}$$

where:

- $\pi_{i,t-1}$ = previous day portfolio weight
- $r_{i,t}$ = stock return

The weights are shifted by one day:

$$\pi_{i,t-1}$$

to avoid look-ahead bias.

This ensures that only information available before day t is used for trading.

3.6 Market Portfolio Returns

The market portfolio return is computed using market weights:

$$R_{m,t} = \sum_{i=1}^N \mu_{i,t-1} r_{i,t}$$

This serves as the benchmark portfolio.

3.7 Cumulative Wealth

Portfolio wealth evolves as:

$$W_t = \prod_{i=1}^t (1 + R_{p,i})$$

Similarly, market wealth is:

$$W_{m,t} = \prod_{i=1}^t (1 + R_{m,i})$$

3.8 Relative Wealth

Relative wealth measures outperformance against the market benchmark.

$$RW_t = \frac{W_t}{W_{m,t}}$$

Interpretation:

- $RW_t > 1$ indicates outperformance
- $RW_t < 1$ indicates underperformance

Relative wealth is a central concept in Stochastic Portfolio Theory because the objective is often to outperform the market portfolio over long horizons.

3.9 Performance Metrics

Several performance metrics are computed to evaluate portfolio performance.

3.9.1 Annualized Arithmetic Return

The average daily portfolio return is:

$$\mathbb{E}[R_p] = \frac{1}{T} \sum_{t=1}^T R_{p,t}$$

where:

- $R_{p,t}$ = portfolio return at time t
- T = total number of trading days

Annualized arithmetic return is:

$$\mu_{annual} = 252 \times \mathbb{E}[R_p]$$

Substituting the expectation term:

$$\mu_{annual} = 252 \times \left(\frac{1}{T} \sum_{t=1}^T R_{p,t} \right)$$

This measures the average annual portfolio return.

3.9.2 Annualized Volatility

Portfolio volatility is computed using the standard deviation of daily returns.

$$\sigma(R_p) = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - \mathbb{E}[R_p])^2}$$

Annualized volatility is:

$$\sigma_{annual} = \sqrt{252} \times \sigma(R_p)$$

Substituting the volatility expression:

$$\sigma_{annual} = \sqrt{252} \times \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - \mathbb{E}[R_p])^2}$$

This measures annualized portfolio risk.

3.9.3 Sharpe Ratio

The Sharpe Ratio measures return per unit of risk.

$$S = \frac{\mu_{annual}}{\sigma_{annual}}$$

Substituting both terms:

$$S = \frac{252 \times \left(\frac{1}{T} \sum_{t=1}^T R_{p,t} \right)}{\sqrt{252} \times \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - \mathbb{E}[R_p])^2}}$$

A higher Sharpe Ratio indicates superior risk-adjusted performance.

3.9.4 Geometric Return

The geometric return measures compounded portfolio growth.

The average log return is:

$$\mathbb{E}[\log(1 + R_p)] = \frac{1}{T} \sum_{t=1}^T \log(1 + R_{p,t})$$

Annualized geometric return is computed as:

$$G = 252 \times \mathbb{E}[\log(1 + R_p)]$$

Substituting the expectation term:

$$G = 252 \times \left(\frac{1}{T} \sum_{t=1}^T \log(1 + R_{p,t}) \right)$$

This measures the average annual compounded growth rate of the portfolio.

4 Parametric Extension of Stochastic Portfolio Theory

The previous SPT model used manually selected diversity parameters such as:

$$p = 0.5$$

and

$$p = 0.8$$

These values remained fixed throughout the entire investment horizon.

A major limitation of this approach is that the portfolio construction rule is manually designed rather than learned from data.

To address this limitation, the Parametric SPT model introduces a trainable parameter:

$$p$$

which is optimized directly using historical market data.

4.1 Learnable Diversity Parameter

The portfolio construction rule remains:

$$\pi_{i,t} = \frac{\mu_{i,t}^p}{\sum_{j=1}^N \mu_{j,t}^p}$$

However, unlike the previous model:

$$p \neq \text{fixed}$$

Instead:

$$p$$

becomes a trainable model parameter.

The optimization algorithm automatically learns the value of p that produces superior relative performance against the market portfolio.

4.2 Relative Return Objective

The previous SPT implementation evaluated portfolio performance directly.

This model instead optimizes **relative performance** versus the market portfolio.

The relative return is:

$$R_{rel,t} = R_{p,t} - R_{m,t}$$

where:

- $R_{p,t}$ = portfolio return
- $R_{m,t}$ = market portfolio return

Positive values indicate outperformance against the market.

4.3 Relative Sharpe Ratio Optimization

The model maximizes the Sharpe Ratio of relative returns.

Expected relative return:

$$\mathbb{E}[R_{rel}] = \frac{1}{T} \sum_{t=1}^T R_{rel,t}$$

Relative return volatility:

$$\sigma(R_{rel}) = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{rel,t} - \mathbb{E}[R_{rel}])^2}$$

Relative Sharpe Ratio:

$$S_{rel} = \frac{\mathbb{E}[R_{rel}]}{\sigma(R_{rel})}$$

The optimization objective becomes:

$$\max_p S_{rel}$$

The implemented loss function is:

$$\mathcal{L} = -\frac{\mathbb{E}[R_{rel}]}{\sigma(R_{rel})}$$

Substituting the full expressions:

$$\mathcal{L} = -\frac{\frac{1}{T} \sum_{t=1}^T R_{rel,t}}{\sqrt{\frac{1}{T} \sum_{t=1}^T (R_{rel,t} - \mathbb{E}[R_{rel}])^2}}$$

4.4 Gradient-Based Learning of p

The parameter p is initialized as:

$$p = 0.5$$

The optimizer computes the gradient:

$$\frac{\partial \mathcal{L}}{\partial p}$$

and updates the parameter iteratively:

$$p \leftarrow p - \eta \frac{\partial \mathcal{L}}{\partial p}$$

where:

- η = learning rate

The optimization is performed using the Adam optimizer.

4.5 Why This Improves Over Classical SPT

The previous SPT model assumes that a manually chosen diversification strength remains optimal permanently.

This assumption is restrictive because different market regimes may favor different portfolio concentration structures.

By learning:

$$p$$

directly from data, the model can automatically discover:

- optimal diversification intensity
- appropriate small-cap exposure
- stronger relative-performance structure

without manual parameter selection.

4.6 Key Difference from Previous SPT Model

Feature	Classical SPT	Parametric SPT
Diversity Parameter p	Fixed Manually	Learned from Data
Optimization	None	Gradient-Based
Adaptivity	Static	Dynamic
Objective	Portfolio Growth	Relative Sharpe Maximization
Learning Mechanism	Absent	Adam Optimization

The main contribution of Parametric SPT is therefore the transition from manually designed portfolio rules to data-driven portfolio rule learning while preserving the theoretical structure of Stochastic Portfolio Theory.

5 Functional Stochastic Portfolio Theory (Functional SPT)

5.1 Motivation and Key Improvement over Previous Models

Previous SPT models used deterministic transformations such as:

$$w_i \propto \mu_i^p$$

where:

- μ_i = market weight of stock i
- p = fixed or learned scalar parameter

Functional SPT generalizes this idea by learning a nonlinear mapping:

$$f : \text{Features} \rightarrow \text{Portfolio Weights}$$

Instead of manually specifying the portfolio generating function, the neural network learns the transformation directly from data.

5.2 Core Idea of Functional SPT

Instead of:

$$w_i \propto \mu_i^p$$

the model learns:

$$w_i \propto f(x_i)$$

where:

- x_i = feature representation of stock i
- $f(\cdot)$ = neural network portfolio generating function

The final normalized weights are:

$$w_i = \frac{f(x_i)}{\sum_{j=1}^N f(x_j)}$$

This preserves:

- positive weights
- fully invested portfolio
- functional portfolio generation

while allowing flexible nonlinear portfolio construction.

5.3 Feature Construction

The feature score is constructed as:

$$x = 0.5 \times \text{Rank}(\mu) + 0.5 \times \text{Rank}(\text{RiskAdjustedMomentum})$$

5.3.1 Risk-Adjusted Momentum

Momentum over 20 trading days:

$$M_{20,i,t} = \frac{P_{i,t}}{P_{i,t-20}} - 1$$

Rolling volatility:

$$\sigma_{20,i,t} = \text{Std}(R_{i,t-19:t})$$

Risk-adjusted momentum:

$$RAM_{i,t} = \frac{M_{20,i,t}}{\sigma_{20,i,t} + \epsilon}$$

This rewards stocks with:

- strong momentum
- lower volatility

simultaneously.

5.4 Cross-Sectional Ranking

Features are converted into percentile ranks:

$$\text{Rank}_{i,t} = \frac{\text{Position of stock } i}{N}$$

The rank transformation amplifies differences between high-ranked and low-ranked stocks.

5.5 GARCH Volatility Estimation

The volatility process is modeled as:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

where:

- ω = long-run volatility component
- α = sensitivity to recent shocks
- β = volatility persistence
- ϵ_{t-1}^2 = previous squared return shock

5.6 Recursive Volatility Update

Volatility is updated recursively:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

This enables the model to adapt dynamically to:

- volatility clustering
- market stress
- changing market regimes

5.7 Neural Network Portfolio Generator

The model learns a smooth portfolio generating function:

$$f(x_i; \theta)$$

where:

- x_i = feature score
- θ = neural network parameters

The network contains:

- hidden nonlinear layers
- residual connections
- Softplus output activation

5.8 Residual Connection

The residual structure is:

$$h(x) = F(x) + x$$

This improves:

- gradient flow
- training stability
- convergence speed

5.9 Softplus Activation

The output layer uses:

$$\text{Softplus}(x) = \log(1 + e^x)$$

This ensures:

$$f(x_i) > 0$$

therefore all portfolio weights remain positive.

5.10 Final Portfolio Weights

After neural-network scoring:

$$w_{i,t} = \frac{f(x_{i,t})}{\sum_{j=1}^N f(x_{j,t})}$$

5.11 GARCH Risk Adjustment

Portfolio weights are adjusted using GARCH volatility ranks:

$$w_{i,t}^{adj} = \frac{w_{i,t}}{1 + \lambda g_{i,t}}$$

where:

- $g_{i,t}$ = GARCH volatility rank
- λ = volatility penalty coefficient

After adjustment:

$$w_{i,t}^{final} = \frac{w_{i,t}^{adj}}{\sum_j w_{j,t}^{adj}}$$

This penalizes highly volatile stocks.

5.12 Objective Function

The optimizer minimizes:

$$\begin{aligned}\mathcal{L} &= -\mathcal{J} + \lambda_{smooth}\mathcal{L}_{smooth} \\ \mathcal{J} &= G_{rel} + \lambda_S S - \gamma D - \lambda_C C - \lambda_{DD} DD\end{aligned}$$

where:

- G_{rel} = relative log-wealth growth

- S = Sharpe ratio term
- D = downside risk penalty
- C = concentration penalty
- DD = drawdown penalty

5.13 Relative Log-Wealth Objective

Core SPT objective:

$$G_{rel} = \mathbb{E} [\log(1 + R_p) - \log(1 + R_m)]$$

Expanded form:

$$G_{rel} = \frac{1}{T} \sum_{t=1}^T [\log(1 + R_{p,t}) - \log(1 + R_{m,t})]$$

where:

$$R_{p,t} = \sum_{i=1}^N w_{t,i} r_{t,i}$$

and

$$R_{m,t} = \sum_{i=1}^N \mu_{t,i} r_{t,i}$$

where:

- $R_{p,t}$ = portfolio return at time t
- $R_{m,t}$ = market portfolio return at time t
- $w_{t,i}$ = portfolio weight of stock i at time t
- $\mu_{t,i}$ = market weight of stock i at time t
- $r_{t,i}$ = return of stock i at time t
- N = number of stocks
- T = total number of trading periods

The market return is computed using market-capitalization-style weights:

$$\mu_{t,i} = \frac{P_{t,i}}{\sum_{j=1}^N P_{t,j}}$$

where $P_{t,i}$ denotes the price of stock i at time t .

The objective therefore maximizes the average relative logarithmic growth of the portfolio compared to the market portfolio.

Using logarithmic wealth is important because:

- log-returns naturally model compounding
- long-term wealth growth becomes additive over time
- relative wealth accumulation versus the market is directly optimized

5.14 Sharpe Ratio Component

The Sharpe component is:

$$S = \frac{\mathbb{E}[R_p]}{\sigma(R_p)}$$

Expanded expectation:

$$\mathbb{E}[R_p] = \frac{1}{T} \sum_{t=1}^T R_{p,t}$$

Portfolio volatility:

$$\sigma(R_p) = \sqrt{\frac{1}{T} \sum_{t=1}^T (R_{p,t} - \bar{R}_p)^2}$$

This rewards stable risk-adjusted performance.

5.15 Downside Risk Penalty

Negative returns are penalized more aggressively:

$$D = \mathbb{E}[\min(R_p, 0)^2]$$

Expanded form:

$$D = \frac{1}{T} \sum_{t=1}^T \min(R_{p,t}, 0)^2$$

This discourages large losses.

5.16 Concentration Penalty

To prevent excessive concentration:

$$C = \mathbb{E} \left[\sum_{i=1}^N w_i^2 \right]$$

Expanded form:

$$C = \frac{1}{T} \sum_{t=1}^T \sum_{i=1}^N w_{i,t}^2$$

Higher concentration produces larger penalties.

5.17 Drawdown Penalty

Portfolio wealth:

$$W_t = \prod_{k=1}^t (1 + R_{p,k})$$

Rolling maximum wealth:

$$M_t = \max_{s \leq t} W_s$$

Drawdown:

$$DD_t = \frac{M_t - W_t}{M_t}$$

Final drawdown penalty:

$$DD = \mathbb{E}[DD_t^2]$$

This penalizes deep portfolio collapses.

5.18 Smoothness Regularization

Second-order finite difference:

$$f''(x) \approx f(x+1) - 2f(x) + f(x-1)$$

Smoothness penalty:

$$\mathcal{L}_{smooth} = \mathbb{E}[(f''(x))^2]$$

This ensures:

- stable weight transitions

- smooth rank-to-weight mapping
- reduced overfitting

5.19 Gradient-Based Optimization

The neural network parameters are updated using Adam optimization.

Parameter update rule:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

where:

- η = learning rate
- θ = all neural network parameters

Gradients are computed using backpropagation.

5.20 Gradient Clipping

To prevent exploding gradients:

$$\|g\| \leq c$$

where:

- g = gradient vector
- c = clipping threshold

This stabilizes deep-network training.

5.21 Learning Rate Scheduling

Cosine annealing warm restarts dynamically adjust the learning rate:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \frac{\pi t}{T} \right)$$

This improves convergence quality.

5.22 Early Stopping

Training stops when objective improvement stagnates.

This prevents:

- overfitting
- unstable late-stage training
- degradation in out-of-sample performance

5.23 Monthly Rebalancing

Portfolio weights are recalculated daily internally but applied with monthly holding periods:

$$w_t = w_{t+1} = \dots = w_{t+20}$$

This reduces:

- turnover
- transaction costs
- excessive trading noise

5.24 Key Improvements over Previous SPT Models

Feature	Classical SPT	Parametric SPT	Functional SPT
Fixed Functional Form	Yes	Semi-flexible	No
Learnable Parameters	No	Single p	Full Neural Network
Uses Momentum	No	No	Yes
Uses Volatility	No	No	Yes
GARCH Risk Control	No	No	Yes
Multi-Objective Optimization	No	Limited	Yes
Drawdown Control	No	No	Yes
Smoothness Constraints	No	No	Yes
Dynamic Risk Penalization	No	No	Yes

5.25 Main Conceptual Difference

Earlier SPT models assumed the portfolio generating function was known beforehand.

Functional SPT instead learns the generating function directly from market data:

$$f(\text{features}) \rightarrow \text{portfolio weights}$$

This transforms classical SPT into a fully data-driven portfolio construction framework while preserving the structural principles of Stochastic Portfolio Theory.

6 Transaction Cost Adjustment

To make the portfolio evaluation more realistic, transaction costs are incorporated during portfolio rebalancing.

Whenever portfolio weights change, trading activity generates execution costs such as:

- brokerage fees
- bid-ask spread costs
- market impact costs

- slippage

These costs reduce realized portfolio returns.

6.1 Portfolio Turnover

Portfolio turnover measures the magnitude of portfolio reallocation between consecutive rebalance periods.

For time t :

$$\text{Turnover}_t = \sum_{i=1}^N |w_{t,i} - w_{t-1,i}|$$

where:

- $w_{t,i}$ = weight of stock i at time t
- N = number of stocks

Higher turnover implies greater trading activity and therefore larger transaction costs.

6.2 Transaction Cost Model

Transaction costs are modeled as a proportional penalty on turnover:

$$\text{Cost}_t = c \times \text{Turnover}_t$$

where:

- c = transaction cost rate

In this implementation:

$$c = 0.001$$

corresponding to a transaction cost of:

$$0.1\%$$

per unit turnover.

6.3 Net Portfolio Return

The transaction-cost-adjusted portfolio return becomes:

$$R_{p,t}^{net} = R_{p,t}^{gross} - \text{Cost}_t$$

where:

- $R_{p,t}^{gross}$ = portfolio return before transaction costs
- $R_{p,t}^{net}$ = portfolio return after transaction costs

6.4 Rebalancing Frequency

The portfolio is rebalanced every:

21 trading days

approximately corresponding to monthly rebalancing.

Transaction costs are therefore applied only on rebalance dates rather than every trading day.

6.5 Impact on Portfolio Performance

Including transaction costs provides a more realistic estimate of deployable strategy performance.

High-turnover strategies may exhibit strong gross returns but weaker net performance after accounting for trading frictions.

Transaction-cost-adjusted evaluation is therefore essential for practical portfolio implementation.